

Optimising Tail Latency of Edge Applications

Shrutya Chawla, Shivankar Srijan Singh, Shamik Sinha, Medha Kashyap

Wireless Systems Engineering Lab
Indraprastha Institute of Information Technology, Delhi

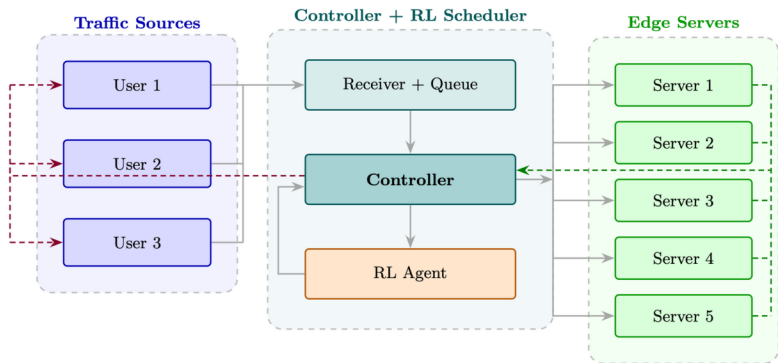
www.iiitd.ac.in

INTRODUCTION

Modern edge applications, such as object detection, speech recognition, and real-time prediction, need very low latency to work correctly. However, edge servers have different CPU/GPU speeds, which leads to unpredictable tail latency and missed deadlines.

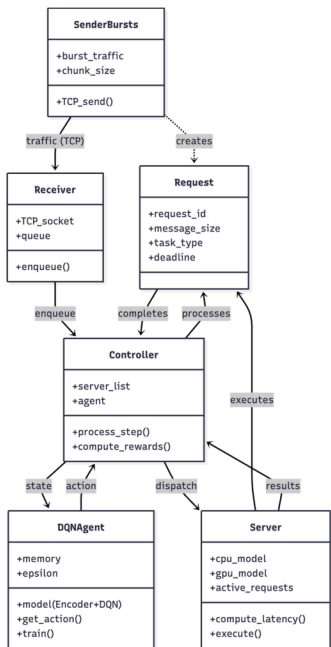
Why does this matter? High tail latency reduces the reliability of latency-critical systems. Traditional schedulers ignore hardware differences, changing workloads, and queueing delays. Even a small number of missed deadlines can make real-time services unusable.

Our approach? We build a reinforcement-learning-based controller that learns how to send requests to heterogeneous servers. It uses a unified state encoder and a dual reward design to improve both short-term load balancing and long-term deadline success.



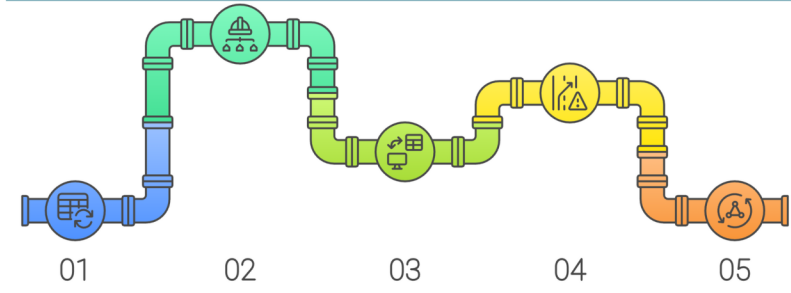
The goal of this work is to reduce tail latency and increase the number of deadline-meeting requests in edge computing systems. This is challenging because servers have different CPU/GPU capabilities, workloads change over time, creating variable states, and queueing delays vary unpredictably with load. The system also operates under practical constraints: 5–7 heterogeneous servers that can run only a limited number of tasks, bursty request patterns, and total latency determined by propagation, transmission, and computation.

SYSTEM OVERVIEW



The diagram depicts the complete request processing pipeline of our RL-based edge scheduling system. SenderBursts generates Request objects with task specifications (detection/speech/prediction) and deadlines, then transmits them via TCP to the Receiver, which deserializes and enqueues batches to the Controller. The Controller serves as the central orchestrator, querying the DQNAgent with the current system state (server loads, queue lengths, request characteristics). The DQNAgent, powered by an integrated State Encoder and DQN neural network, selects an optimal subset of heterogeneous edge servers using an ϵ -greedy policy learned through experience replay. The Controller then dispatches requests to the chosen servers, which execute tasks and return results along with latency measurements. These results feed back to the Controller's reward computation module, which calculates step rewards (load balancing minus waiting time) and episodic rewards (discounted step sum plus degree of satisfaction), enabling the DQNAgent to continuously improve its scheduling policy through reinforcement learning.

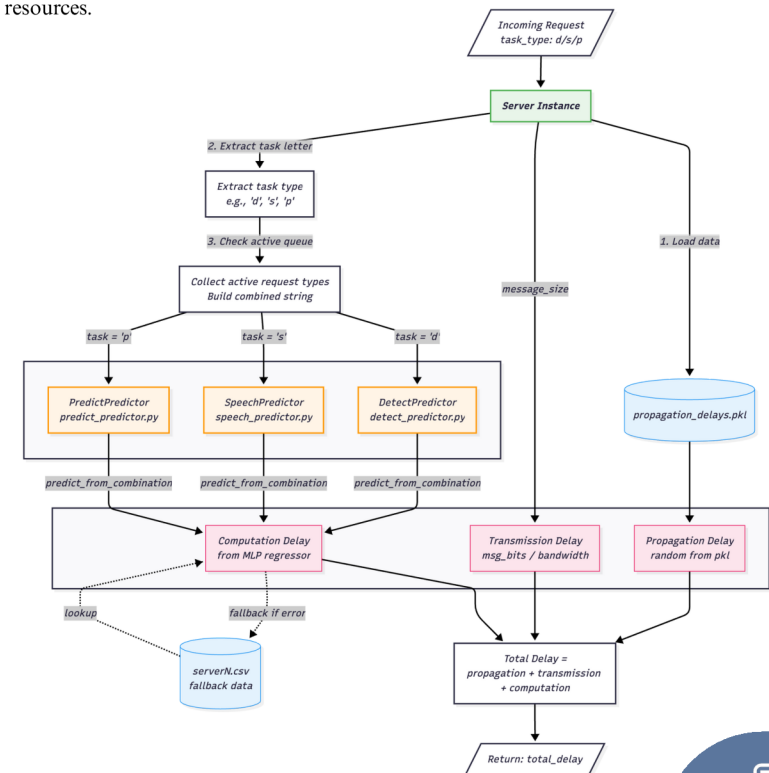
METHODOLOGY & IMPLEMENTATION



Our methodology centers on building a reinforcement-learning scheduler that learns how to route requests across heterogeneous edge servers. The RL agent uses a state composed of flattened request and server features, such as message size, bandwidth, queue lengths, CPU/GPU usage, hardware specifications, and timing information, while accounting for the fact that the state size changes with the number of servers, through a learned state encoder that maps heterogeneous server states into a fixed 32-dimensional vector. Latency is modelled through queueing delay estimates, propagation delay, transmission delay, and computation delay, which are predicted by task-specific MLP regressors for detection, speech, and prediction tasks.

Server

The server module simulates end-to-end request execution, including network delays and computation delays, while supporting the parallel execution of multiple requests. It provides an experimental framework to evaluate latency behaviour under realistic network and workload conditions. Each of the 5 servers had different computational resources.



Server	CPU Model	Cores	GPU Model	VRAM	RAM
Server 1	Intel Core i5-12600K (12th Gen)	16	RTX 3090	24 GB	32 GB
Server 2	AMD Ryzen 5 7600X	6	–	–	12 GB
Server 3	AMD Ryzen 5 5600GT	12	–	–	32 GB
Server 4	Intel Core i7-11700 (11th Gen)	16	–	–	16 GB
Server 5	AMD Ryzen 9 7950X	32	RTX 4080	16 GB	32 GB

The Server has 5 responsibilities.

- Parallel Request Scheduling:** Runs up to five requests in parallel and tracks active tasks, timing, and queue rejections.
- Propagation Delay Estimation:** Samples one-way delay from a dataset built using multi-location ping measurements.
- Transmission Delay Calculation:** Computes uplink and downlink transmission times from message size and bandwidth.
- Computation Delay Simulation:** Uses task-specific ML regressors to estimate processing time and adjusts for concurrency effects.

Regressor Training Pipeline

For each server, three task-specific datasets were used: Speech, Predict, and Detect. The following ML models were trained for every dataset:

- Linear Regression (with feature scaling)
- Random Forest Regressor
- Gradient Boosting Regressor

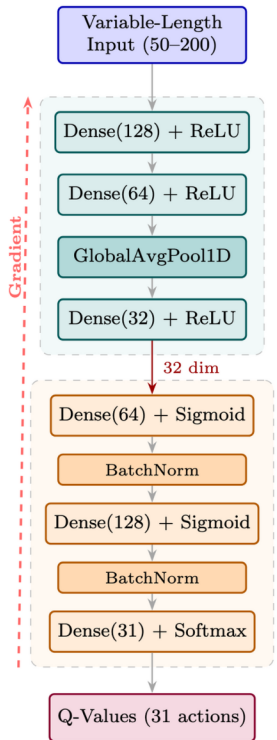
Each model was evaluated using RMSE on a held-out test set. The model with the lowest RMSE was selected as the final regressor for that task on that server.

Encoder + DQN Agent

The encoder–DQN architecture enables RL scheduling in a heterogeneous edge environment by converting variable-length server and request features into a fixed 32-dimensional state. A small encoder captures system load and hardware differences, while the DQN layers map this encoded state to all possible server-subset actions. The entire model trains end-to-end, allowing the encoder to learn task-specific representations. An epsilon-greedy strategy guides exploration, gradually shifting from random actions to learned scheduling decisions.

Step and Episodic Reward Architecture

The reward system was redesigned so the controller computes both step and episodic rewards. Step rewards give immediate feedback based on load balance and waiting time, while episodic rewards combine discounted step rewards with overall deadline satisfaction. Centralising this logic in the controller provides full visibility into server loads and queue states. During training, each process step stores experiences with step rewards, and after every episode, the accumulated transitions receive an episodic reward before being used for replay. The current satisfaction metric is a placeholder and will be refined with better deadline tracking and hyperparameter tuning.



Controller (Central Orchestrator)

The Controller coordinates end-to-end scheduling, RL decision-making, and training across all edge servers. It manages a queue of incoming request batches, queries the DQN agent for server-selection actions, dispatches requests to heterogeneous servers, and collects experiences for step and episodic reward computation. Acting as the bridge between the Receiver and the Servers, it processes each request chunk as a training step and triggers episodic training after fixed intervals. Its 2 key responsibilities:

- Request Processing:** Handles request batches, updates system state, and queries the agent for actions.
- Server Management:** Tracks server availability and dispatches requests to chosen subsets.

Receiver (M/M/1 Queueing Interface)

The Receiver is a TCP-based queueing module that accepts request chunks, models bounded-buffer M/M/1 behavior, and forwards validated data to the

Controller. It runs asynchronously, enforces a strict queue limit, deserializes incoming numpy arrays, and provides backpressure by rejecting new connections when the buffer is full, effectively shaping network traffic into controllable batches. Responsibilities:

- TCP Handling:** Accepts connections, receives data, and manages timeouts.
- Queue Discipline:** Queue limits and accept windows like an M/M/1 system.
- Controller Handoff:** Forwards decoded request chunks to controller. send_to_server().

RESULTS & CONCLUSION

The system now runs end-to-end with an integrated encoder–DQN scheduler, heterogeneous server simulation, M/M/1 queueing, latency regressors, and full TCP-based communication. The agent shows early learning behavior by avoiding overloaded servers, and the encoder successfully handles variable-length states. Converting dictionaries and lists to NumPy arrays further reduced processing overhead and improved throughput. Overall, the prototype confirms that RL-based scheduling with learned state encoding is feasible and functional, though current results use simplified (dummy) reward functions and require full training for deeper evaluation.

FUTURE WORK

- Refine the reward structure and implement a proper satisfaction metric.
- Train on real workloads and evaluate convergence by plotting training and testing plots.
- Improve network modeling and latency realism.
- Compare RL performance against baselines to verify tail-latency optimization.



Read Our Full
Project Report
Here.

Author
Information.

Shrutya Chawla : 2022487
Shivankar Singh : 2022479
Shamik Sinha : 2022559
Medha Kashyap : 2022292

Advisor :
Dr. Arani Bhattacharya
PhD Mentor :
Ms. Jyoti Shokhanda

Contact
Information.
Reach out to us at:
[first name][roll no.].@iiitd.ac.in